



Module 20-1

Functional Design Challenges (2)

cādence[®]

In this section, we're talking about the challenges with functional design.

We've already learned a little bit about the different kinds of hard hardware constructs that we see and the different levels of abstraction at which we have to write code.

In order to describe those, this section is going to talk about some of the problems you'll find.

Objective

In this module, you will:

- Identify the physical factors that are present challenges to functional design.

Topics include:

- Low-power requirements – clock gating
- Introduction to low-power schemes (power intent using UPF language)



Your objective is to get started using SystemVerilog to describe the behavior of a digital design. To do that, you need to know some fundamental SystemVerilog language constructs.

Mainly they're related to physical factors that present challenges to the functional design.

Although we're coding at the lower level of abstraction register transfer level, we can't ignore the physical factors which will occur later on in the process.

So, we can see here a list of the topics we talk about.

And following this will be an introduction to low-power schemes, which use a language known as UPF.



Module 20-2

Clock Gating

cādence®

This page does not contain notes.

Objective

In this module, you will:

- Explore the concept of clock gating.



This page does not contain notes.

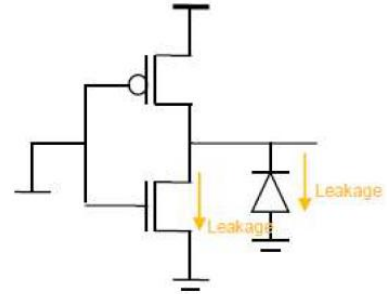
Power Consumption in a CMOS Digital IC

There are two sources of power consumption in CMOS ICs:

- Static power
- Dynamic power

Static Power – Leakage

- This is constant.
- It depends on the IC process technology we use, a function of:
 - The Supply Voltage (V_{dd})
 - The Switching Threshold (V_{th})
 - The transistor dimensions
- We can't do anything about this in our RTL code.



Clock gating is a way of reducing power consumption.

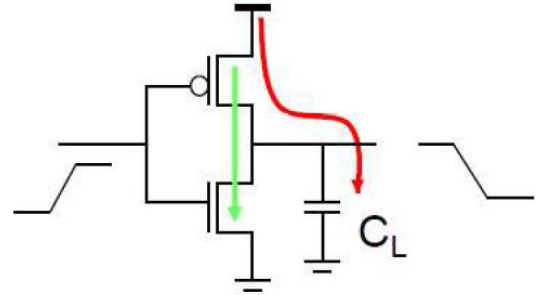
In a CMOS chip, which all modern chips are, there's two sources of power consumption. One is static power; one is dynamic static power. You can't really do anything about this in your RTL code. This is based upon the technology, and it's constant, hence its name. It's a function of the supply voltage, the switching threshold, the kind of technology you've got, and the transistor dimensions. So, there's nothing you can do about that. That's all part of the process.

Dynamic Power Consumption in a CMOS Digital IC

Dynamic Power – Switching

A sum of:

- Switching power - **the red line.**
 - Charging/discharging capacitance of nets and transistors
- Short circuit power – **the green line.**
 - During switching, both P and N (CMOS) transistors are on for a very short time.
 - That is, the power supply is short-circuited.



We can do something about Dynamic Power in our RTL code.

- Our RTL determines when switching occurs.



What happens when you do this to a car battery?

What you can do something about, however, is dynamic power because this is due to switching.

The switching power is the sum of the charging and discharging of the capacitance of the load, the load being the inputs to other transistors on the wire, the interconnects. This green line shows you the short circuit power. CMOS is a complementary transistor. When one is on, the other is off, and vice versa. Now, during the switchover, when you're switching something from on to off, both transistors are on for a very brief period.

What happens if you do that to a car battery? Well, you draw like infinite current, in effect, until everything just burns down and melts.

We can do something about that in our code because we can determine how often the switching occurs.

What Can We Do in RTL to Reduce Power?

- Use RTL coding techniques, for example
 - Manual resource sharing
 - Gray coding for FSM state encoding.
- However, these techniques might be time-consuming to implement and not have a significant enough effect to make it worth the effort.
- Reducing the switching activity has a much greater effect.
- We need to ensure that switching only occurs when absolutely necessary.
- Remember, we said in a previous lecture that a complex processor at a 7nM node can only have 50% of the transistors active at any given time to keep below the power budget.
- To reduce the switching activity, we use a technique called **Clock Gating**.



We can use techniques we've discussed already, like manual resource sharing, Gray coding for state machines, etc. But these might be time-consuming to implement and verify and not really have enough effect to make it worth the bother. What has a much greater effect is, reducing the switching activity so that we only have parts of the circuit switching when they need to.

Remember, we said previously that for a new process node like seven nanometers, you might only be allowed to switch 50 percent of the transistors at any given time if you keep below the power budget and stop the chip kind of overheating.

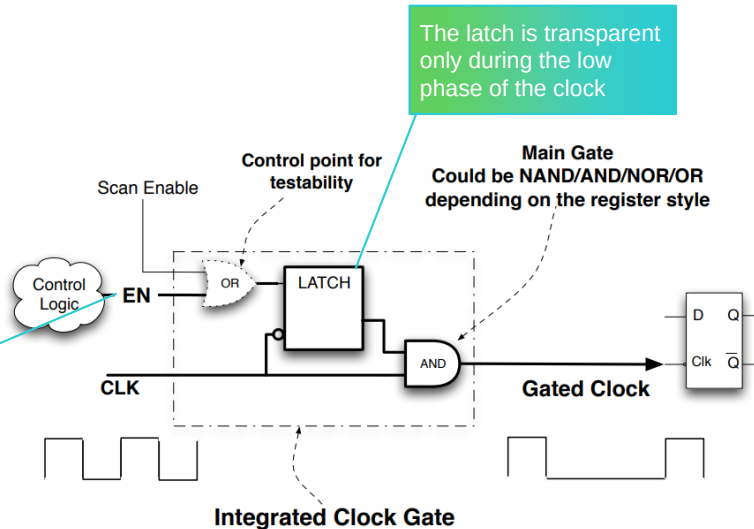
We can reduce switching activities by a technique called clock gating. Clock gating is a good technique but it creates another verification headache for us.

What Is Clock Gating?

A technique used in synchronous circuits to reduce dynamic power by removing the clock when it is not essential to design intent functionality.

- Effectively, we prune the clock tree to save power.
- However, remember the PPA tradeoff?
- This is at the expense of more gates.

Design Block
Enable Signal
generated from our
Control Logic



8

© Cadence Design Systems, Inc. All rights reserved.

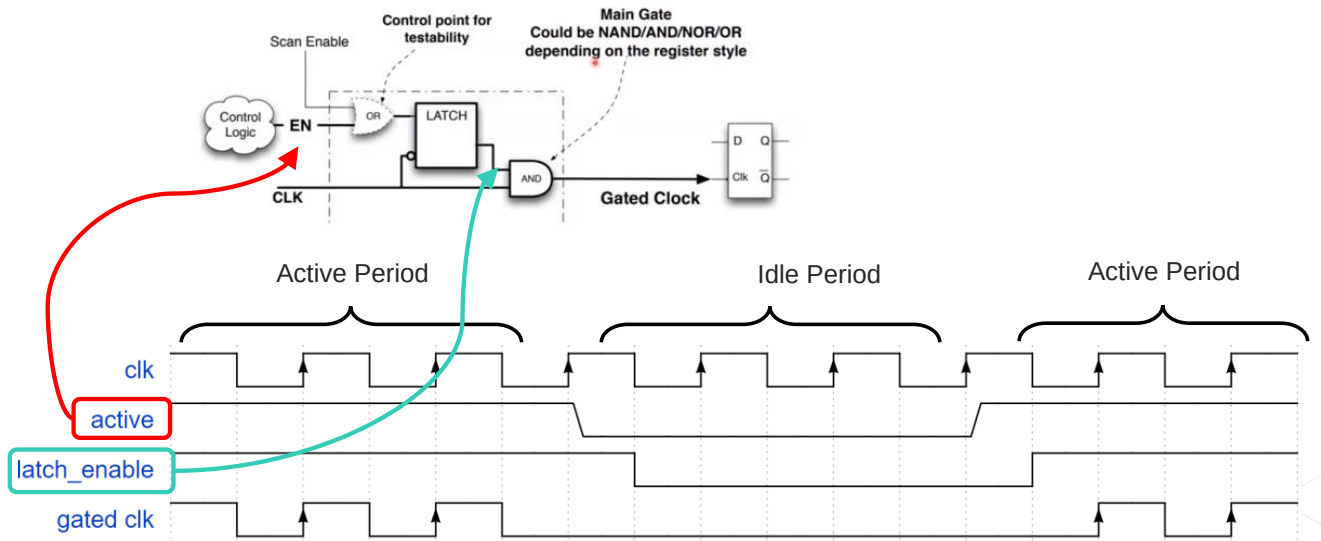


Clock gating is a technique to reduce dynamic power by effectively pruning the clock tree.

Remove the clock when it's not essential to the design functionality. This is at the expense of more gates; however, we make a higher performance area in the tradeoff. We have more gates to consume less power. Here's an example of clock gating. The clock to our chip isn't coming from one of the input pins. It's coming from all this different logic here.

We can see some control logic here, and we've got an and gate here with the clock, which is coming from the input pin and ending with all these different signals. The control logic for our clock gating and other things like scan enable is used for manufacture testing. It's a control point for testability. We've got all this extra circuitry to reduce the number of times we switch this flip-flop.

Example: Waveforms for a Gated Clock



This page does not contain notes.

Why Is Clock Gating Such a Hard Problem?

The need for power efficiency requires clock gating to be manually defined at the RTL level.

Because doing it in RTL is more efficient than:

- Software-level clock gating.
- Automated clock gating applied by synthesis tools.

Because we code the clock gating manually in RTL:

- It's highly error prone.
- It makes verification much more difficult.

To obtain a reasonable amount of coverage of clock gating activity using simulation-based flows may be unfeasible in acceptable timescales.

The exhaustive nature of formal verification fits well into this domain.



Clock gating is a complex problem. You can't do this automatically.

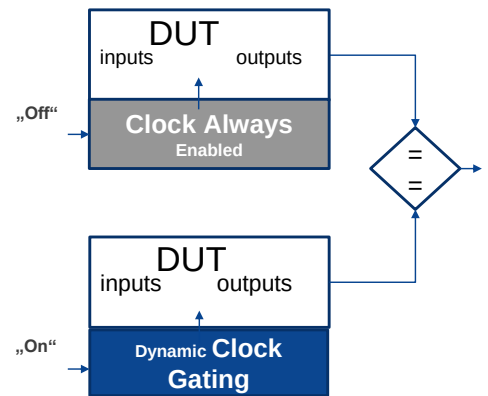
There are things in the software that can apply such techniques, so you can get synthesis tools that use Clock gating automatically or do this at some kind of software level. Still, you're never going to be the efficiency of a human. This is why people do it because you get the best power efficiency. This institution has its own problems because it's a person doing it. It's highly error-prone, and it makes verification much harder now.

We now must consider all the different stages or states the clock gating could be at.

Doing this in a simulation is unfeasible in any kind of acceptable timescale, being as we've got this deadline the tapeout to meet. This problem is well suited to a formal because its formal and exhaustive nature allows it to determine that the clock gating works correctly under all conditions.

What Happens When We Change a Design to Use Gated Clocks?

- We may not have been aware that we had a low power requirement when we first created our design.
- Clock gating is intended only to reduce power consumption, not change the functionality of the design.
- This greatly complicates our verification requirements.
- We can solve this with a formal technique known as “Equivalence Checking.”
- We ask a formal tool to prove to us that the original design is the same as the design modified to have clock gating.
- Cadence JasperGold has an Equivalence Checking App called SEC (Sequential Equivalence Checking) to verify that clock gating does not affect the original functionality.



What happens if we change your design to use gated clocks? We had a previous design that didn't consider clock gating. Now we have decided we need clock gating because we're using this newer technology where we can't switch everything at the same time. We introduced clock gating to reduce power consumption, not to change the functionality of the design. So how can we deal with this in formal?

We can do this with something known as equivalence checking so we can take the design that didn't have a clock gating gated clock. On a design that does have a click-gated clock and check that the outputs are always identical, i.e., we didn't change the functionality by introducing clock gating. This app is known as sequential equivalence checking in just four gold. That's one of its primary uses for inserting clock gating didn't change the functionality of the design.

Can We Define All Low-Power Requirements in RTL?

No, we can't.

Other low-power techniques related to physical effects have to be defined using a different language.

Because RTL has no way of expressing those techniques.

The Power Intent Language is called UPF (Unified Power Format).

UPF is an IEEE standard – IEEE 1801.

This is the subject of the next lecture.



Can we define all our power requirements in RTL? Can we address all those low-power requirements?
No, we can't.

We need other low-power techniques related to the physical effects, and we define these using a different language. This is the thing like isolation cells, level shifting cells in terms of shifting the voltage levels. We can't describe that in RTL because it doesn't have the semantics to do it. So, we need another language known as Unified Power Format (UPF), which specifies the power intent. This is not really a standard. Watch IEEE 1801.



Module 20-3

Low-Power Concepts

cādence[®]

This page does not contain notes.

Objectives

In this module, you will:

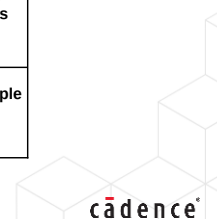
- List power reduction techniques.
- Analyze low-power concepts.



This page does not contain notes.

Power Reduction Techniques Summary

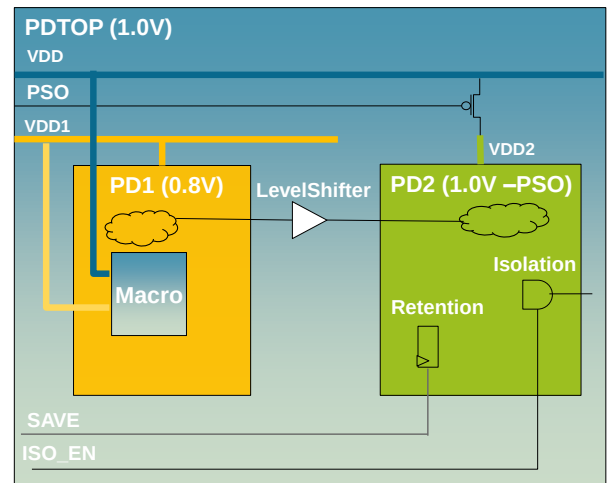
Power Reduction Technique	Leakage Power	Dynamic Power	Timing	Area Penalty	Methodology Impact	Methodology Change
Low-Power Optimization	10%	10%	0%	10%	None	None
Multi-Vt	6X	0%	0%	0%	Low	Multi-Vt library needed
Clock Gating	0%	20%	0%	<2%	Low	Clock-gating cells needed and extra overhead in STA
Multi-Supply Voltage (MSV)	2X	40-50%	0%	<10%	Medium	Micro-architecture and methodology needs to be domain aware; need voltage regulators and level shifters; verification and analysis challenge
Power Shut-Off (PSO)	10-50X	0%	4-8%	5-15%	Medium-High	Insertion of switch cells; retention flops; wake-up and shut-down time analysis; power shut off and restore verification
Dynamic Voltage Frequency Scaling (DVFS)	2-3X	40-70%	0%	<10%	High	Deterministic scheduling; multi-mode optimization and analysis flow needed; clock synchronization
Substrate Biasing	10X	0%	10%	<10%	High	Maintain well separation; multiple power rail distribution; static timing analysis



This page does not contain notes.

Low-Power Concepts

Power Domain	A user-defined grouping of instances that share the same supply nets.
Level Shifters	Handle interfaces that have different voltage levels.
Power Switches	Switch definition that turns on/off power to a domain.
Isolation	- Isolates a power shutoff domain from functional logic. - Ensures electrically and functionally correct interfaces during shutoff.
Retention	- Ability to specify registers as being maintained through power shutoff. - Many styles of control are available.
Macro Model	Hard IPs with multiple supplies that have been blackboxed e.g., memory models.



Three Power Domains: PDTOP (VDD + VSS)
PD1 (VDD1 + VSS), PD2 (VDD2 + VSS)

The main aspects of power intent for low-power design are Power Domain, Level Shifters, Power Switches, Isolation, Retention, and Macro Model.

What Are the Low-Power Library Cells Required for Implementing the PSO Technique?

PSO design requires the following low-power cells:

- Power switches to turn on/off the power supply to a domain.
- Isolation cell is used to isolate an active (ON) domain from a signal from an inactive (OFF) domain.
- Retention cell is a special D flip-flop that has control to save a state when power is turned OFF so the unit can recover when power is restored.
- AON buffers for signals that feed through the shutoff domain.



Answer

PSO design requires the following low-power cells:

- Power switches to turn on/off power supply to a domain.
- Isolation cell is used to isolate an active (ON) domain from a signal from an inactive (OFF) domain.
- Retention cell is a special D flip-flop that has control to save a state when power is turned OFF so the unit can recover when power is restored.
- AON buffers for signals that feedthrough shutoff domain.



Module 20-4

Introduction to Low-Power Simulation

cā dence®

This page does not contain notes.

Objective

In this module, you will:

- Identify implicit low-power simulation behaviors.

Topics include:

- Power-shutoff simulation behavior
 - Port isolation, state retention, state loss
- The generic power control sequence
 - For balloon latch technologies: edge-based or level-based save and restore
 - For master/slave-alive latch technologies: a retention condition

Before studying this module, you need to be familiar with Xcelium™ digital simulation and debug.



Your objective is to identify implicit low-power simulation behaviors, that is, port isolation, state retention, and power shutoff.

This training module discusses:

- Power-shutoff simulation behavior, including port isolation, state retention, and state loss; and
- The generic power control sequence, which for balloon latch technologies, can use edge-based or level-based save and restore signals, and for master/slave-alive latch technologies, uses a retention condition.

Glossary of Low-Power Simulation Fundamental Terms

Low-Power Simulation (LPS)	Power-aware digital simulation.
Power Design	Unique power structure.
Power Domain	Collection of design elements sharing a power distribution network.
Power Shutoff (PSO)	Feature to switch domain power on and off.
Isolation	Defines logic signal level while otherwise not driven due to power shutoff.
Retention	Preserves state values during shutoff.
State Retention Power Gating (SRPG)	Retaining states through power shutoff.
Balloon Latch	Dedicated low-power state-retention latch.



This training module introduces terms that may be new to you.

- **Low-Power Simulation (LPS)** is power-aware digital simulation.
- A **Power Design** is a unique power structure that can be associated with either a top design or with one or more logic modules.
- A **Power Domain** is a collection of instances, pins and ports that can share the same power distribution network.
- **Power Shutoff (PSO)** is a power reduction technique that switches off a power domain while it is not active.
- **Isolation** is a technique that defines logic signal values while the signals are otherwise not driven due to power shutoff.
- **Retention** is a technique that preserves sequential element states that would otherwise be lost due to power shutoff.
- **State Retention Power Gating** is a technique to retain design states through power shutoff.
- A **Balloon Latch** is a dedicated always-on low-power latch added to a sequential element to preserve the element value while shutoff.

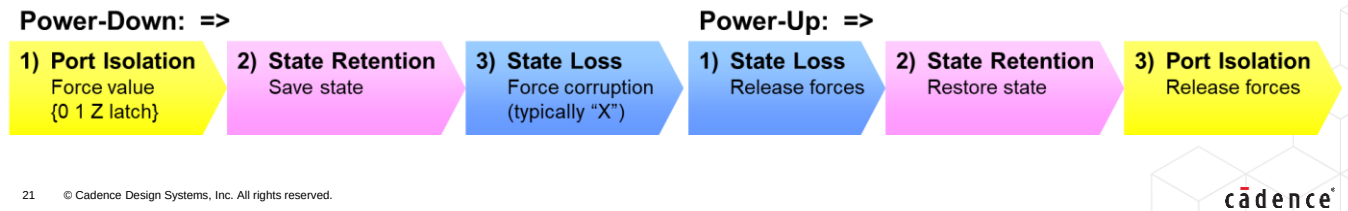
What Is Low-Power Simulation?

Low-power simulation is the simulator mimicking the target hardware's reduced-power behavior.

You simulate reduced-power behavior to confirm design reaction to power-change events.

For example, for power shutoff, the simulator mimics the behaviors:

- Port isolation – forces a known logic level onto a powered-on input not driven due to power shutoff.
- State retention – saves state values through power shutoff and then restores them.
- State loss – forces the unknown logic level onto elements driven by the powered-down domain.



21 © Cadence Design Systems, Inc. All rights reserved.

Digital simulation performs three behaviors related to power domain shutoff.

For the power-down sequence, it:

1. Forces a specified value onto the staying-on receivers of nets driven by the domain about to be shut off, to prevent propagating unknown and, thus, potentially destructive values from a power domain that is shut off to a power domain that remains on.
2. Saves the value of selected sequential elements in the domain about to be shut off; and
3. Mimics state loss in the shut-off domain by corrupting all domain inputs that have loads in the domain, and all domain state elements and driven nets that are not part of an always-on cell.

For the power-up sequence, it:

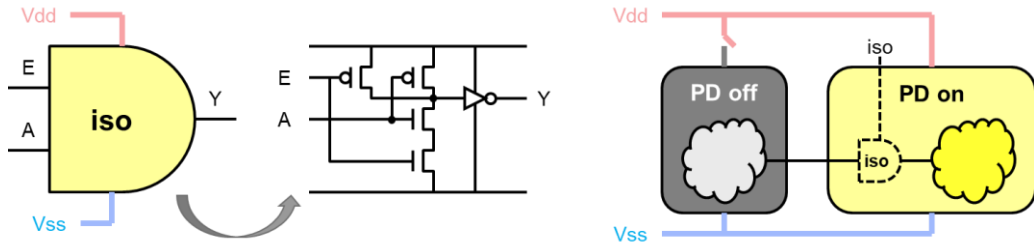
1. Releases the corrupting forces;
2. Restores the saved values of the selected sequential elements; and
3. Releases the isolation forces.

What Is Port Isolation in a Low-Power Simulation Context?

Port isolation forces a logic level onto an input pin not driven due to power shutoff.

You isolate a powered-on cell input driven by powered-off cell output to prevent rogue input values from affecting the powered-on cell functionality and possibly even damaging it.

- For gates, simulation executes behaviors specified by the technology library isolation cell simulation model.
- For RTL, simulation forces specified values onto undriven inputs of cells in the powered-on power domains.



22 © Cadence Design Systems, Inc. All rights reserved.

cadence

A power domain is usually connected to other power domains. When shut off, it could propagate unknown and thus potentially destructive levels to power domains that remain on. Low-power design adds isolation logic between power domains that can shut off and power domains that can remain on, to prevent propagating unknown states from a power domain that is shut off to a power domain that remains on.

The post-synthesis gate-level netlist includes actual isolation cells to provide the isolation behavior.

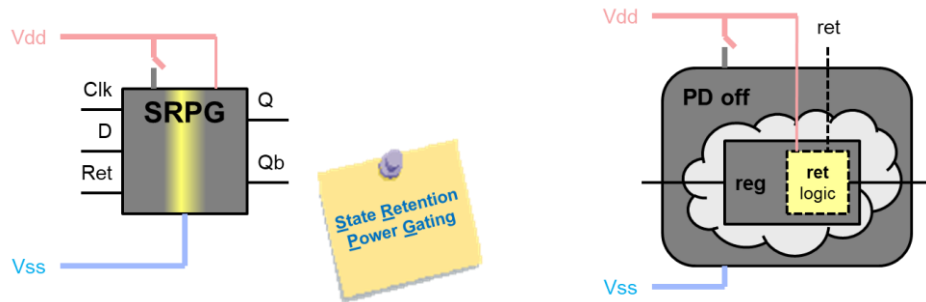
At the register-transfer level, the simulator implicitly models this pin isolation by simulating the effect of inserting an appropriate isolation cell. It inserts the virtual isolation cell outside the powered-off domain as a driver of the net connected to the powered-on domain. The isolation logic drives a specified logic value to the pin in the powered-on domain.

What Is State Retention in a Low-Power Simulation Context?

State retention saves state values through power shutoff and then restores them.

You retain critical (not all) states through power shutoff to resume operation after power up.

- For gates, simulation executes behaviors specified by the technology library retention cell simulation model.
- For RTL, simulation retains and restores states of specified state elements.



23 © Cadence Design Systems, Inc. All rights reserved.

cadence

When a power domain is powered down, the states of key sequential elements in the power domain, such as some latches, flip-flops, and registers, must be saved and retained for the entire shutoff period. When the power domain is powered back up, the saved states must be restored to the sequential elements. To ensure that a powered-down domain resumes normal operation after power up, state retention cells replace these key sequential elements. Power-efficient designs apply state retention to only the key sequential elements that must be retained.

The post-synthesis gate-level netlist includes actual state retention cells to provide the state retention behavior.

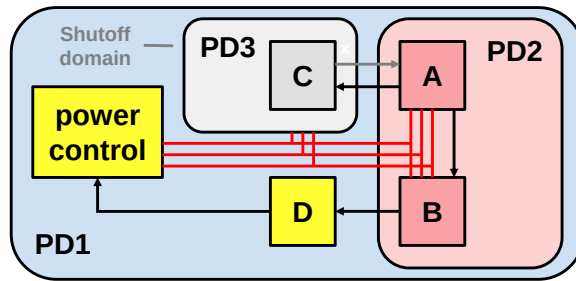
At the register-transfer level, the simulator implicitly models state retention by simulating the effect of inserting appropriate state retention cells. It saves the state of the cell upon the save event and restores the state of the cell upon the restore event. If the save or restore signals meanwhile become unknown, then the simulator makes the saved value also unknown.

What Is State Loss in a Low-Power Simulation Context?

State loss corrupts state values while power is shut off.

Outputs of unpowered cells float to undetermined values. The simulation must mimic this behavior.

- For gates, simulation executes behaviors specified by the technology library retention cell simulation model.
- For RTL, simulation corrupts by default all domain inputs that have loads in the domain and corrupts all nets, signals, and variables driven by the powered-down domain except those specified not to be corrupted.



24 © Cadence Design Systems, Inc. All rights reserved.



At the physical level, when a power domain shutoff condition becomes true, appropriate switches open, resulting in a loss of voltage to the domain. With no proper biasing, the local voltages float, and the output of every cell in the shutoff region is undetermined.

Simulation models this behavior by corrupting all[†] domain inputs that have loads in the domain, and all domain state elements and driven nets that are not part of an always-on cell. It forces the output of the corrupted elements to a corruption value. All signals within the shut off region transition to the corruption value, and all sequential elements (registers) and variables within the shutoff region lose their state and transition to the corruption value.

The simulator also powers off the associated power domains if the shutoff condition becomes unknown.

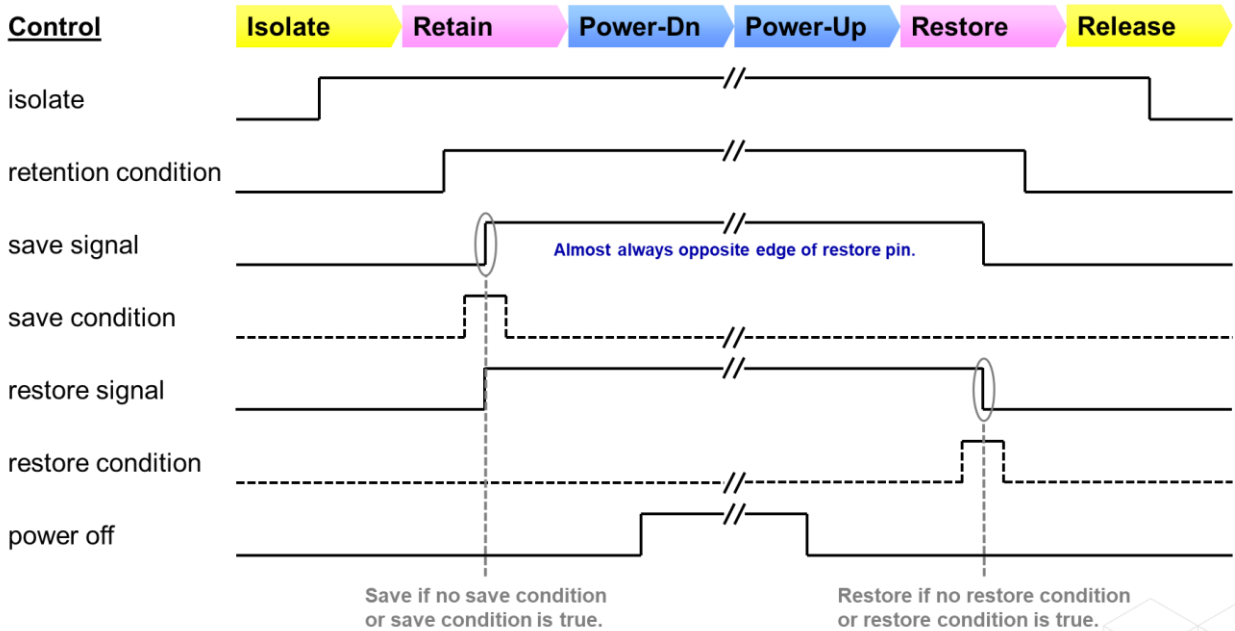
Upon shutting off a SystemVerilog power domain, the simulator assigns the unknown logic value (X) to all variables within the shutoff domain and drives all nets within the shutoff domain that are not part of an always-on cell to the unknown logic value.

Upon shutting off a VHDL power domain, the simulator assigns the unknown logic value (X) to all signals and variables of standard logic types within the shutoff domain. To signals and variables of enumerated types, it, by default, assigns the leftmost value.

When a power domain is again powered on, the simulator releases the forced corruption and resumes normal simulation activity. Power-up behavior can generate glitches in domain signal values similar to those that occur when real hardware powers up. The simulator by default does not restore any previously forced values, but you can optionally change this behavior.

[†] An optional driver-based algorithm more accurately chooses the loads in the domain to corrupt.

Generic Power Cycle Sequence for Balloon-Latch: Edge-Based



25 © Cadence Design Systems, Inc. All rights reserved.

cadence

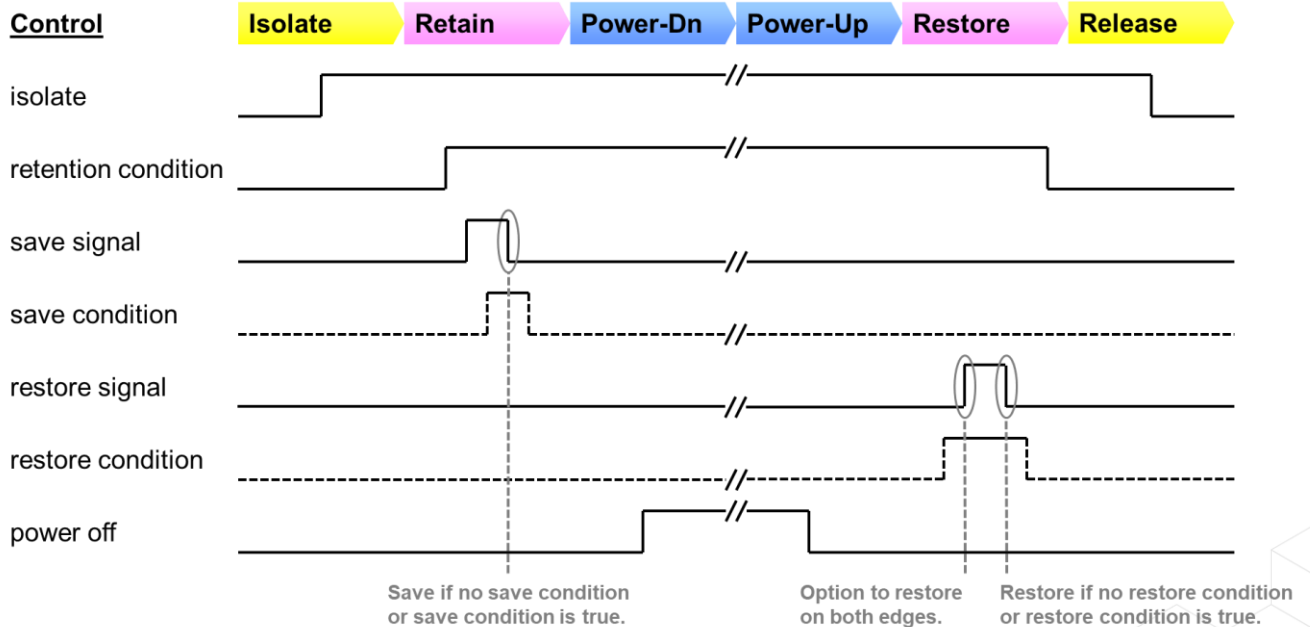
Implicit simulation behaviors support multiple retention technologies.

This is the generic balloon latch edge-based state retention technology.

- Isolation introduction is independent of the retention technology.
- State retention occurs on the save signal active edge.
 - The save condition is optional. You can use it to guard the save edge.
- The optional retention condition, if false, corrupts the saved value.
- Power-down and power-up are independent of the retention technology and do not affect balloon latch edge-based state retention.
- State restoration occurs on the restore signal active edge.
 - The restore condition is optional. You can use it to guard the restore edge.
- Isolation removal is independent of the retention technology.
- The save and restore signals are almost always opposite edges of one cell pin.

You describe all these behaviors by entering commands into your power intent document.

Generic Power Cycle Sequence for Balloon-Latch: Level-Based



26 © Cadence Design Systems, Inc. All rights reserved.

cadence

Implicit simulation behaviors support multiple retention technologies.

This is the generic balloon latch level-based state retention technology.

- Isolation introduction is independent of the retention technology.
- State retention occurs on the save signal trailing edge.
 - The save condition is optional. You can use it to guard the save edge.
- The optional retention condition, if false, corrupts the saved value.
- Power-down and power-up are independent of the retention technology and do not affect balloon latch level-based state retention.
- State restoration occurs on the restored signal trailing edge.
 - The restore condition is optional. You can use it to guard the restore edge.
 - You can optionally specify to perform state restoration on both restore signal edges.
- Isolation removal is independent of the retention technology.

You describe all these behaviors by entering commands into your power intent document.

For Xcelium simulation, the elaborator option `-lps_restore_level` specifies restoring the state on both edges of the level-sensitive restore signal.

Generic Power Cycle Sequence for Master-Slave Latch

Control

Isolate

Retain

Power-Dn

Power-Up

Restore

Release

isolate

retention condition

power off

save
(option)

save
(default)

restore

27 16
© Cadence Design Systems, Inc. All rights reserved.

cadence

Implicit simulation behaviors support multiple retention technologies.

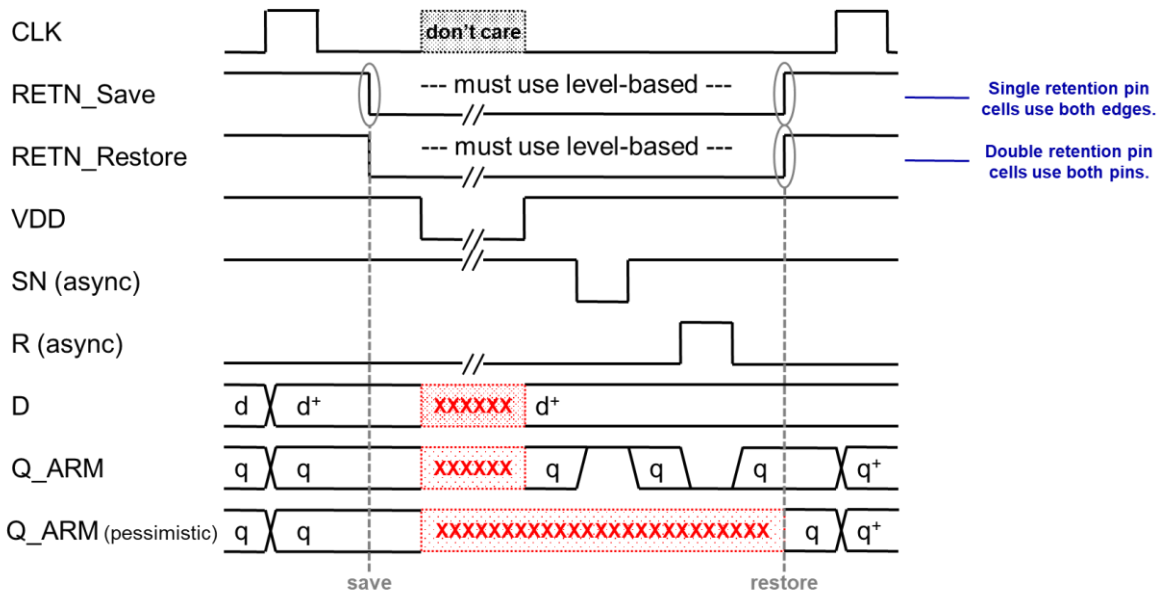
This is the generic master/slave-alive state retention technology.

- Isolation introduction is independent of the retention technology.
- State retention optionally occurs upon the retention condition becoming true.
- Power-down is independent of the retention technology. The slave latch, by default, stays alive, and so retains its state.
- State restoration occurs upon power-up.
- Isolation removal is independent of the retention technology.

You describe all these behaviors by entering commands into your power intent document.

For Xcelium™ simulation, the elaborator option `-lps_rtn_save_lock` specifies to instead save the state at the retention condition leading edge.

Non-Standard Level-Based Variation for ARM® State Retention



```
set_design_attributes -elements {retention_strategy_list} -attribute <ret_cell_type arm_retn_cell_1[p] | ret_cell_type arm_retn_cell_2>
```

28 © Cadence Design Systems, Inc. All rights reserved.

cadence

Some ARM state retention cells exhibit a slightly different behavior from the IEEE Std. 1801-2015 retention strategies cannot describe. They restore state continually when the clock is low, and power is restored, and up to the end of the restoration interval. Meanwhile, the set and reset pins can combinatorially affect the current output but not the saved state. Xcelium simulation provides an RTL solution for SystemVerilog and VHDL that supports these ARM state retention cells. For these cells, it supports only level-sensitive retention rules and issues an error upon encountering an edge-sensitive retention rule. To designate the retention strategies to which this behavior applies, set their retention-cell-type (ret_cell_type) attribute, for single-pin save and restore, to the value arm-retention-cell-1 (arm_retn_cell_1), and for double-pin save and restore, to the value arm-retention-cell-2 (arm_retn_cell_2). The single-pin save and restore, but not the double-pin save and restore, also has a pessimistic cell model that holds the cell output unknown until the end of the restoration interval.

```
-----
set_design_attributes -elements {MY_RR_RULE} -attribute ret_cell_type arm_retn_cell_1
set_design_attributes -elements {MY_RR_RULE} -attribute ret_cell_type arm_retn_cell_1p
set_design_attributes -elements {MY_RR_RULE} -attribute ret_cell_type arm_retn_cell_2
```

Test Your Understanding

- What basic power behaviors can a simulator add to an RTL simulation?
 - To an RTL simulation, the simulator adds implicit port isolation and state retention, corruption, and restoration behaviors.
- What control signals does a power controller typically generate?
 - A power controller typically generates isolation, retention, and shutoff signals.
- Explain how you differently use edge-based and level-based balloon-latch state retention.
 - Edge-based balloon-latch retention cells almost always have only one save-restore pin and use opposite save-restore active edges. Level-based balloon-latch retention cells more frequently have separate save-restore pins and, by default, operate on their inactive edges.



This page does not contain notes.

Summary

In this module, you:

- Identified implicit low-power simulation behaviors.

This training module discussed:

- Power-shutoff simulation behavior.
 - Port isolation, state retention, state loss.
- The generic power control sequence.
 - For balloon-latch technologies: edge-based or level-based save/restore.
 - For master-slave latch technologies: a retention condition.



You should now be able to identify implicit low-power simulation behaviors.

This training module discussed:

- Power-shutoff simulation behavior, including port isolation, state retention, and state loss; and
- The generic power control sequence, which for balloon latch technologies, can use edge-based or level-based save and restore signals, and for master/slave-alive latch technologies, uses a retention condition.